

Raspberry Pi Kubernetes Cluster Plan

k3s on 4× Pi 5 (8GB), growing to 5 — single control plane, homelab + learning

TL;DR

Run a **single k3s server + k3s agents** on every other node. Phase 1 is **1 server + 3 agents** (4 Pis you have today). When the 5th Pi arrives, just join it as a 4th agent — **1 server + 4 agents**. No HA, no rebuild. Run Ubuntu Server 24.04 arm64 on every node. Put the control-plane role on whichever Pi has the best storage (NVMe HAT > USB SSD > SD card) — etcd writes will hammer it.

Tradeoff you're accepting: if the control-plane Pi dies, the API is unreachable until you restore it. Workloads on workers keep running, but you can't deploy, scale, or schedule until the CP is back. Mitigate with regular etcd snapshots (covered at the end).

Topology

Phase 1 — 4 nodes (today)

Role	Count	Hostname	k3s role	Notes
Control plane	1	rpi-control	server	Best storage; runs API, scheduler, etcd
Worker	3	rpi-node{1..3}	agent	Run workloads; SD OK, SSD better

Phase 2 — 5 nodes (when Pi #5 arrives)

Role	Count	Hostname	k3s role	Notes
Control plane	1	rpi-control	server	Unchanged from Phase 1
Worker	4	rpi-node{1..4}	agent	Just add the 5th Pi as a new agent

Why this shape

- **Single CP, by choice.** One server + many workers is the simplest possible cluster: minimal moving parts, no quorum to reason about, no etcd leader elections to debug. Ideal for learning kubectl, manifests, helm, ingress, and storage without HA noise on top.
- **Failure mode is well-defined.** Worker death = pods reschedule onto surviving workers. CP death = API down until you fix the CP Pi; running workloads on workers keep serving. Acceptable for homelab use.
- **Pi 5 8GB is comfortable.** A k3s server idles around 600–900 MB. Plenty of headroom for the control plane plus a few light workloads on the same node if you want to taint it as schedulable.
- **k3s vs kubeadm.** For a Pi homelab, k3s wins: single binary, embedded etcd, Traefik + ServiceLB + local-path-provisioner preinstalled. You still get vanilla kubectl, Helm, manifests — it's real Kubernetes.
- **Growth path stays trivial.** Adding the 5th Pi (or a 6th, 7th...) is just one curl command on that node. No cluster reshape needed.

Hardware role assignment

Match the storage to the role. etcd is write-heavy and SD cards die fast under that load — months, not years.

Storage on Pi	Best role	Why
NVMe HAT	Control plane	Best write endurance + latency for etcd; pick this one as CP
USB SSD	Worker w/ persistent volumes	Good for stateful pods, DBs, Longhorn replicas
SD card only	Pure worker	Fine for stateless pods; avoid PV writes

Network plan

- Wired Gigabit on every node. WiFi is fine for one-off experiments but wreaks havoc on etcd heartbeats and pod-to-pod latency.
- All nodes on the same /24 subnet. Pin each node to a stable IP via DHCP reservation on your router (easier than static configs per node).
- Reserve a small block of IPs on your LAN for LoadBalancer services (e.g., 10 IPs). k3s's built-in **ServiceLB** works on day 1; swap in **MetalLB** when you want real LB semantics.

Example IP plan

```
192.168.1.10  rpi-control      (control plane / NVMe Pi)
192.168.1.11  rpi-node1       (worker)
192.168.1.12  rpi-node2       (worker)
192.168.1.13  rpi-node3       (worker)
192.168.1.14  rpi-node4       (worker – the Pi #5 once it arrives)
192.168.1.200-210 reserved for LoadBalancer service IPs
```

Build runbook (Ansible)

Two-phase flow. Flashing has to happen per SD card (can't SSH to a Pi that hasn't booted yet), so 01-flash-pi.sh stays. Everything after first boot is one Ansible playbook that hits all 4 Pis in parallel and is idempotent — rerun it freely.

Ansible files live in ~/git/rpi-cluster/ansible/:

File	Purpose
ansible.cfg	Defaults — uses inventory.ini, 10 forks, SSH ControlMaster for speed.
inventory.ini	Maps hostnames to .local addresses and groups them as k3s_server and k3s_agent.
group_vars/all.yml	Variables: k3s install args, kubeconfig path, Rancher hostname + bootstrap password (override on the CL
site.yml	One playbook with 5 plays: OS prep, k3s server, k3s agents, fetch kubeconfig, install Rancher. Use --tags

Step 0 — One-time setup on your laptop

```
sudo apt -y install ansible
ansible-galaxy collection install kubernetes.core community.general

# helm is needed by the Rancher play:
curl -fsSL https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
```

Step 1 — Flash + boot each Pi

Per SD card (still scripted, not Ansible):

```
# rpi-control first, then each worker:
sudo ~/git/rpi-cluster/scripts/01-flash-pi.sh rpi-control /dev/mmcblk0
sudo ~/git/rpi-cluster/scripts/01-flash-pi.sh rpi-node1 /dev/mmcblk0
sudo ~/git/rpi-cluster/scripts/01-flash-pi.sh rpi-node2 /dev/mmcblk0
sudo ~/git/rpi-cluster/scripts/01-flash-pi.sh rpi-node3 /dev/mmcblk0
```

Boot each Pi, wait ~90 s for cloud-init. Verify SSH works to all:

```
cd ~/git/rpi-cluster/ansible
ansible all -m ping
# Expect 'pong' from rpi-control + rpi-node{1..3}.
```

Step 2 — Run the full playbook

```
cd ~/git/rpi-cluster/ansible

# Everything except Rancher (OS prep, k3s server, k3s agents, kubeconfig):
ansible-playbook site.yml --skip-tags rancher

# Then Rancher (override the default bootstrap password first):
ansible-playbook site.yml --tags rancher \
  -e rancher_bootstrap_password='something-long-and-yours'
```

What happens: Ansible SSHes to all 4 Pis in parallel, runs apt upgrade + swap off + iscsid, asserts memory cgroups are enabled (and stops if not). Then it installs k3s server on rpi-control, grabs the join token off the filesystem, and installs k3s agent on every worker — joining them automatically with the right URL + token. No copy/paste of tokens between steps. Then it pulls the kubeconfig to ~/.kube/config-homelab on your laptop.

Useful tag combinations

```
# Smoke-test SSH only:
ansible all -m ping

# Just the OS prep on every node (e.g., after kernel update):
ansible-playbook site.yml --tags prep

# Just a single host (e.g., re-provision rpi-node2):
ansible-playbook site.yml --limit rpi-node2

# Just refetch kubeconfig:
ansible-playbook site.yml --tags kubeconfig

# Dry-run everything (show what would change, don't apply):
ansible-playbook site.yml --check --diff
```

Step 3 — Adding the 5th Pi later

Uncomment the rpi-node4 line in inventory.ini, flash its card, boot, then:

```
ansible-playbook site.yml --skip-tags rancher --limit rpi-node4
```

Done. Ansible re-runs the prep + agent install on just that host. Existing nodes are skipped because the playbook is idempotent.

Installing Rancher Manager

What it is. Rancher Manager is a web UI for managing one or more Kubernetes clusters — workload browser, kubectl-in-browser, RBAC, monitoring, Fleet (GitOps), and one-click app installs. It runs *inside* the cluster as a Deployment in the cattle-system namespace.

What the playbook does. The rancher tag runs locally on your laptop (uses the kubeconfig already fetched to ~/.kube/config-homelab) and Helm-installs three things in order: **ingress-nginx** (since Traefik was disabled), **cert-manager** (Rancher needs it for TLS), then **Rancher** itself.

Hostname trick. Rancher needs a hostname to serve from. To avoid editing /etc/hosts on every device, the playbook defaults to rancher.<rpi-control-ip>.nip.io — *nip.io* is a wildcard DNS service that resolves any x.y.z.w.nip.io back to its embedded IP. Override with -e rancher_hostname=rancher.example.lan if you prefer a real DNS name.

Resource cost. Rancher + nginx + cert-manager together use about 1.5–2 GB RAM. Pi 5 8GB handles it comfortably.

Run it (after the rest of the playbook has succeeded):

```
cd ~/git/rpi-cluster/ansible
ansible-playbook site.yml --tags rancher \
  -e rancher_bootstrap_password='something-long-and-yours'

# When the play finishes, the final debug task prints the URL to open.
```

First page load shows a self-signed cert warning — accept it, then set a real admin password on the welcome screen.

If something gets stuck

```
# Watch the Rancher pods come up:
kubectl -n cattle-system get pods -w

# Most common slow step: pulling arm64 images on first install (5-10 min).

# If the URL won't load, check the nginx LoadBalancer got an IP:
kubectl -n ingress-nginx get svc ingress-nginx-controller
# EXTERNAL-IP should be one of the rpi-control / rpi-node{1..3} IPs.

# Logs:
kubectl -n cattle-system logs deploy/rancher --tail=200
```

Importing the cluster into Rancher. Rancher auto-imports the cluster it runs on (it shows up as 'local'). To add additional clusters later, use the 'Import Existing' flow in the UI and run the import command it gives you on the new cluster.

Backups: etcd snapshots

Single-CP means the cluster API depends on rpi-control. Take regular etcd snapshots and store them off-Pi so you can rebuild fast if the SD/SSD dies:

```
# One-shot snapshot (run on rpi-control):
sudo k3s etcd-snapshot save --name manual-$(date +%F)

# Or enable automatic snapshots — every 6 hours, keep last 10.
# Edit /etc/systemd/system/k3s.service and append to ExecStart:
#   --etcd-snapshot-schedule-cron '0 */6 * * *' \
#   --etcd-snapshot-retention 10
```

```
# then: sudo systemctl daemon-reload && sudo systemctl restart k3s

# Snapshots land in /var/lib/rancher/k3s/server/db/snapshots/
# Rsync them to off-Pi storage on a cron from your laptop:
rsync -av victord@rpi-control:/var/lib/rancher/k3s/server/db/snapshots/ ~/backups/k3s/
```

Suggested add-ons for a homelab

Install these once the cluster is healthy. All run fine on Pi 5 8GB.

Add-on	Purpose	Notes
MetalLB	LoadBalancer IPs on bare metal	Or use k3s built-in ServiceLB to start
cert-manager	Auto TLS certs (Let's Encrypt)	Pairs with Traefik or nginx-ingress
Longhorn	Distributed block storage / PVs	Wants SSDs; replicate across 3 nodes
ArgoCD or Flux	GitOps — declare cluster state in git	Great learning vehicle
kube-prometheus-stack	Metrics + Grafana	Heavier; watch RAM on CPs
Pi-hole / AdGuard	Network DNS sinkhole	Classic first homelab workload
Home Assistant	Home automation	Use a StatefulSet with PV on Longhorn

Gotchas worth knowing up front

- **SD card death.** etcd + container logs murder SD cards. If your CP is on SD, plan to migrate to SSD within weeks, not months.
- **Power.** Pi 5 wants the official 27 W USB-C PD supply. Undervoltage throttling on a CP looks like random API timeouts.
- **Cooling.** Active cooling (the official fan or a case with one) is basically required for sustained K8s workloads on Pi 5.
- **iptables-nft vs legacy.** Ubuntu 24.04 uses nft by default and k3s handles it correctly; only a problem if you mix in custom iptables rules.
- **Mixed-arch images.** Always pull arm64 image variants. Most popular images are multi-arch now, but some niche ones aren't.
- **Backups matter more with single CP.** Set up etcd snapshots from day 1 (see Phase 2 section). With no HA, a snapshot is the difference between a 10-minute restore and rebuilding everything from scratch.
- **Don't taint the CP unschedulable** unless you start running out of capacity. With 1 CP + 3 workers, letting light system pods land on the CP keeps cluster utilization sensible.

Appendix A: SD card flash script

Lives in /home/victord/git/rpi-cluster/scripts/. Used once per Pi before Ansible can reach it.

01-flash-pi.sh

Flashes Ubuntu 24.04 to the SD card and writes cloud-init user-data (hostname + your SSH key) + cgroup params to cmdline.txt. Run locally with sudo.

```
#!/usr/bin/env bash
# Flash Ubuntu Server 24.04 arm64 to an SD card and preconfigure it for headless K8s.
# Writes hostname, your SSH key, and the cgroup_memory kernel param.
#
# Usage:    sudo ./01-flash-pi.sh <hostname> <device>
# Example:  sudo ./01-flash-pi.sh rpi-control /dev/mmcblk0
#
# Optional env overrides:
#   USERNAME      (default: the invoking user)
#   SSH_KEY_FILE   (default: ~/.ssh/id_ed25519.pub, falls back to id_rsa.pub)
#   TIMEZONE       (default: system tz)
#   IMAGE_CACHE    (default: ~/.cache/pi-images)

set -euo pipefail

HOSTNAME_ARG="${1:-}"
DEVICE="${2:-}"

if [[ -z "$HOSTNAME_ARG" || -z "$DEVICE" ]]; then
    echo "usage: sudo $0 <hostname> <device>"
    echo "example: sudo $0 rpi-control /dev/mmcblk0"
    exit 1
fi

if [[ $EUID -ne 0 ]]; then
    echo "must run as root (use sudo) – needs to write to $DEVICE"
    exit 1
fi

INVOKER="${SUDO_USER:-$USER}"
USERNAME="${USERNAME:-$INVOKER}"
TIMEZONE="${TIMEZONE:-$(timedatectl show -p Timezone --value 2>/dev/null || echo UTC)}"
IMAGE_CACHE="${IMAGE_CACHE:-/home/$INVOKER/.cache/pi-images}"

# Resolve SSH key
if [[ -z "${SSH_KEY_FILE:-}" ]]; then
    for f in "/home/$INVOKER/.ssh/id_ed25519.pub" "/home/$INVOKER/.ssh/id_rsa.pub"; do
        [[ -f "$f" ]] && SSH_KEY_FILE="$f" && break
    done
fi
if [[ -z "${SSH_KEY_FILE:-}" || ! -f "$SSH_KEY_FILE" ]]; then
    echo "no SSH public key found – set SSH_KEY_FILE=/path/to/key.pub"
    exit 1
fi
SSH_KEY=$(cat "$SSH_KEY_FILE")

# Sanity-check the device
if [[ ! -b "$DEVICE" ]]; then
    echo "$DEVICE is not a block device"
    exit 1
fi
DEVICE_SIZE=$(lsblk -bdno SIZE "$DEVICE")
DEVICE_MODEL=$(lsblk -dno MODEL,TRAN "$DEVICE" | tr -s ' ')
DEVICE_SIZE_H=$(numfmt --to=iec --suffix=B "$DEVICE_SIZE")

echo
echo "=====
echo " FLASH PLAN"
echo "=====
echo "   Hostname:  $HOSTNAME_ARG"
echo "   Device:    $DEVICE ($DEVICE_SIZE_H, $DEVICE_MODEL)"
echo "   Username:  $USERNAME"
echo "   SSH key:   $SSH_KEY_FILE"
echo "   Timezone:  $TIMEZONE"
```

```

echo "   Cache:      $IMAGE_CACHE"
echo "=====
echo
echo "!! ALL DATA ON $DEVICE WILL BE ERASED !!"
read -rp "Type the device path again to confirm ($DEVICE): " CONFIRM
if [[ "$CONFIRM" != "$DEVICE" ]]; then
    echo "abort: device path did not match"
    exit 1
fi

mkdir -p "$IMAGE_CACHE"
chown "$INVOKER:" "$IMAGE_CACHE"

# Find the latest Ubuntu 24.04 arm64 raspi image
echo
echo "[1/5] resolving latest Ubuntu Server 24.04 arm64 image..."
INDEX_URL="https://cdimage.ubuntu.com/releases/noble/release/"
IMG_NAME=$(curl -sf "$INDEX_URL" \
    | grep -oE 'ubuntu-24\.[0-9]+\.[0-9]+-preinstalled-server-arm64\+raspi\.img\.xz' \
    | sort -V | tail -1)
if [[ -z "$IMG_NAME" ]]; then
    echo "could not resolve image name from $INDEX_URL – check connectivity"
    exit 1
fi
IMG_URL="$INDEX_URL$IMG_NAME"
IMG_PATH="$IMAGE_CACHE/$IMG_NAME"
RAW_PATH="${IMG_PATH%.xz}"
echo "   image: $IMG_NAME"

# Download if not cached
if [[ ! -f "$RAW_PATH" ]]; then
    if [[ ! -f "$IMG_PATH" ]]; then
        echo "[2/5] downloading $IMG_URL"
        curl -L --progress-bar -o "$IMG_PATH.partial" "$IMG_URL"
        mv "$IMG_PATH.partial" "$IMG_PATH"
    else
        echo "[2/5] using cached $IMG_PATH"
    fi
    echo "[3/5] decompressing..."
    unxz --keep "$IMG_PATH"
else
    echo "[2/5] using cached $RAW_PATH"
    echo "[3/5] decompression skipped"
fi

# Unmount any auto-mounted partitions
echo "[4/5] unmounting any existing partitions on $DEVICE..."
for p in $(lsblk -lnpo NAME "$DEVICE" | tail -n +2); do
    umount "$p" 2>/dev/null || true
done

# Flash
echo "[5/5] writing image to $DEVICE (this takes 5-15 minutes)..."
dd if="$RAW_PATH" of="$DEVICE" bs=4M conv=fsync status=progress
sync

# Reread partition table and wait for partitions
partprobe "$DEVICE"
sleep 3
BOOT_PART=$(lsblk -lnpo NAME "$DEVICE" | sed -n '2p')
if [[ -z "$BOOT_PART" ]]; then
    echo "could not find boot partition on $DEVICE after flash"
    exit 1
fi

# Mount boot partition
MNT=$(mktemp -d)
mount "$BOOT_PART" "$MNT"
echo
echo "boot partition mounted at $MNT"

# 1) cmdline.txt: append cgroup params if not already there
CMDLINE="$MNT/cmdline.txt"
if [[ -f "$CMDLINE" ]]; then
    if ! grep -q 'cgroup_memory=1' "$CMDLINE"; then

```



```

    # cmdline.txt must stay a single line
    sed -i 's|$| cgroup_memory=1 cgroup_enable=memory|' "$CMDLINE"
    echo " + appended cgroup params to cmdline.txt"
else
    echo " = cmdline.txt already has cgroup params"
fi
fi

# 2) user-data: write cloud-init config
cat > "$MNT/user-data" <<EOF
#cloud-config
hostname: $HOSTNAME_ARG
manage_etc_hosts: true

users:
  - name: $USERNAME
    groups: [adm, sudo]
    shell: /bin/bash
    sudo: ALL=(ALL) NOPASSWD:ALL
    ssh_authorized_keys:
      - $SSH_KEY

ssh_pwauth: false
chpasswd:
  expire: false

timezone: $TIMEZONE
package_update: true
package_upgrade: false

runcmd:
  - swapoff -a
  - sed -i '/ swap / s/^#/' /etc/fstab
EOF
echo " + wrote user-data (hostname=$HOSTNAME_ARG, user=$USERNAME)"

sync
umount "$MNT"
rmdir "$MNT"

# Eject so the user can pull the card cleanly
eject "$DEVICE" 2>/dev/null || true

echo
echo "=====
echo " DONE"
echo "=====
echo " Card ready. Insert in the Pi, power on, wait ~90s, then:"
echo " ssh $USERNAME@$HOSTNAME_ARG.local"
echo "=====

```

Appendix B: Ansible files

All Ansible files live in `/home/victord/git/rpi-cluster/ansible/`.

ansible.cfg

Defaults: uses `inventory.ini`, 10 forks, SSH ControlMaster.

```
[defaults]
inventory = ./inventory.ini
host_key_checking = False
forks = 10
stdout_callback = yaml
callbacks_enabled = profile_tasks
deprecation_warnings = False
retry_files_enabled = False
roles_path = ./roles
collections_path = ~/.ansible/collections

[ssh_connection]
pipelining = True
ssh_args = -o ControlMaster=auto -o ControlPersist=60s -o StrictHostKeyChecking=accept-new
```

inventory.ini

Hosts + groups (k3s_server, k3s_agent).

```
# Inventory for the rpi k3s homelab.
# Edit hostnames / IPs to match your DHCP reservations.

[k3s_server]
rpi-control  ansible_host=rpi-control.local

[k3s_agent]
rpi-node1    ansible_host=rpi-node1.local
rpi-node2    ansible_host=rpi-node2.local
rpi-node3    ansible_host=rpi-node3.local
# rpi-node4  ansible_host=rpi-node4.local  # uncomment when Pi #5 arrives

[k3s_cluster:children]
k3s_server
k3s_agent

[k3s_cluster:vars]
ansible_user=victord
ansible_python_interpreter=/usr/bin/python3
```

group_vars/all.yml

Cluster-wide variables — k3s args, kubeconfig path, Rancher defaults. Override on the CLI with `-e KEY=VAL`.

```
---
# Cluster-wide variables. Override per host/group as needed.

# k3s
k3s_install_url: "https://get.k3s.io"
k3s_version: "" # empty = latest stable. Pin like "v1.31.0+k3s1" once you've chosen one.
k3s_server_args: "--write-kubeconfig-mode 644 --disable traefik"

# Where the kubeconfig from rpi-control should land on your laptop.
kubeconfig_local_path: "{{ lookup('env', 'HOME') }}/.kube/config-homelab"

# Address that ends up in the kubeconfig + the k3s agent join URL.
# Default = the CP's inventory hostname (e.g. rpi-control.local). Override with an
# IP or a real DNS name if .local mDNS isn't reliable on your network. Examples:
#   cluster_endpoint: 192.168.1.10
#   cluster_endpoint: rpi-control.lan
cluster_endpoint: "{{ hostvars[groups['k3s_server'][0]].ansible_host
| default(groups['k3s_server'][0]) }}"

# Rancher
rancher_namespace: cattle-system
rancher_bootstrap_password: "rancher-init-change-me" # override on the CLI: -e rancher_bootstrap_password=...
```

```

rancher_hostname: ""          # empty = auto-compute as rancher.<rpi-control-ip>.nip.io
rancher_chart_version: ""    # empty = latest stable

```

site.yml

Five plays: prep, k3s server, k3s agents, fetch kubeconfig, Rancher.

```

---
# Full provisioning playbook for the rpi k3s homelab.
#
# Usage:
#   ansible-playbook site.yml                # full run
#   ansible-playbook site.yml --tags prep    # OS prep only
#   ansible-playbook site.yml --tags k3s     # prep + k3s install
#   ansible-playbook site.yml --tags rancher # Rancher install only
#   ansible-playbook site.yml --limit rpi-node1 # one host only
#
# First-time prerequisites on your laptop:
#   sudo apt install -y ansible
#   ansible-galaxy collection install kubernetes.core community.general
#   ansible -i inventory.ini all -m ping      # smoke test SSH

- name: Base OS prep on every Pi
  hosts: k3s_cluster
  become: true
  gather_facts: true
  tags: [prep, k3s, all]
  tasks:
    - name: apt update + safe upgrade
      ansible.builtin.apt:
        update_cache: true
        upgrade: safe
        cache_valid_time: 3600

    - name: Install required packages
      ansible.builtin.apt:
        name:
          - curl
          - ca-certificates
          - open-iscsi
          - nfs-common
          - avahi-daemon          # so .local hostnames resolve between Pis
          - libnss-mdns
        state: present

    - name: Manage /etc/hosts entries for every cluster node
      # Drops an Ansible-managed block on every Pi that maps rpi-control,
      # rpi-node1, etc. to their LAN IPs. Robust even if mDNS / DNS flake.
      ansible.builtin.blockinfile:
        path: /etc/hosts
        marker: "# {mark} ANSIBLE MANAGED rpi-cluster"
        block: |
          {% for host in groups['k3s_cluster'] %}
          {{ hostvars[host].ansible_default_ipv4.address }} {{ host }} {{ host }}.local
          {% endfor %}

    - name: Enable iscsid (for future Longhorn)
      ansible.builtin.systemd:
        name: iscsid
        enabled: true
        state: started

    - name: Disable swap (runtime)
      ansible.builtin.command: swapoff -a
      changed_when: false

    - name: Disable swap (persistent in /etc/fstab)
      ansible.builtin.replace:
        path: /etc/fstab
        regexp: '^(^#)\..*\sswap\s.*)$'
        replace: '#\1'

    - name: Verify memory cgroups are enabled
      ansible.builtin.command: awk '/^memory/ {print $4}' /proc/cgroups
      register: cgroup_check

```

```

changed_when: false

- name: Fail if memory cgroup is not enabled
  ansible.builtin.assert:
    that: cgroup_check.stdout == '1'
    fail_msg: >-
      memory cgroup not enabled on {{ inventory_hostname }}.
      Add 'cgroup_memory=1 cgroup_enable=memory' to /boot/firmware/cmdline.txt and reboot.

- name: Install k3s server on rpi-control
  hosts: k3s_server
  become: true
  tags: [k3s, k3s-server, all]
  tasks:
    - name: Check if k3s binary already present
      ansible.builtin.stat:
        path: /usr/local/bin/k3s
        register: k3s_bin

    - name: Run k3s server install
      ansible.builtin.shell: |
        curl -sfl {{ k3s_install_url }} | \
        {% if k3s_version %}INSTALL_K3S_VERSION={{ k3s_version }} {% endif %}sh -s - server {{ k3s_server_args }}
      args:
        executable: /bin/bash
        when: not k3s_bin.stat.exists
        register: k3s_install
        changed_when: k3s_install.rc == 0

    - name: Wait for node-token file
      ansible.builtin.wait_for:
        path: /var/lib/rancher/k3s/server/node-token
        timeout: 120

    - name: Read node-token
      ansible.builtin.slurp:
        src: /var/lib/rancher/k3s/server/node-token
        register: k3s_token_b64

    - name: Stash token as a fact (visible to other plays via hostvars)
      ansible.builtin.set_fact:
        k3s_node_token: "{{ k3s_token_b64.content | b64decode | trim }}"

    - name: Wait for k3s API to report node Ready
      ansible.builtin.command: k3s kubectl get node {{ inventory_hostname }} -o jsonpath='{.status.conditions[?(@.type=="Ready')]'
      register: cp_ready
      until: cp_ready.stdout == "True"
      retries: 30
      delay: 5
      changed_when: false

- name: Install k3s agents on all worker Pis
  hosts: k3s_agent
  become: true
  tags: [k3s, k3s-agent, all]
  tasks:
    - name: Check if k3s already installed
      ansible.builtin.stat:
        path: /usr/local/bin/k3s
        register: k3s_bin

    - name: Run k3s agent install + join
      ansible.builtin.shell: |
        curl -sfl {{ k3s_install_url }} | \
        K3S_URL=https://{{ cluster_endpoint }}:6443 \
        K3S_TOKEN={{ hostvars[groups['k3s_server'][0]].k3s_node_token }} \
        {% if k3s_version %}INSTALL_K3S_VERSION={{ k3s_version }} {% endif %}sh -
      args:
        executable: /bin/bash
        when: not k3s_bin.stat.exists
        register: k3s_join
        changed_when: k3s_join.rc == 0

- name: Fetch kubeconfig to your laptop
  hosts: k3s_server

```

```

become: true
tags: [kubeconfig, all]
tasks:
  - name: Copy /etc/rancher/k3s/k3s.yaml off the Pi
    ansible.builtin.fetch:
      src: /etc/rancher/k3s/k3s.yaml
      dest: "{{ kubeconfig_local_path }}"
      flat: true

  - name: Rewrite server URL to reach the Pi from the laptop
    delegate_to: localhost
    become: false
    ansible.builtin.replace:
      path: "{{ kubeconfig_local_path }}"
      regexp: 'server: https://127.0.0.1:6443'
      replace: "server: https://{{ cluster_endpoint }}:6443"

  - name: chmod 600 on kubeconfig
    delegate_to: localhost
    become: false
    ansible.builtin.file:
      path: "{{ kubeconfig_local_path }}"
      mode: '0600'

- name: Install Rancher Manager (ingress-nginx + cert-manager + Rancher)
  hosts: localhost
  connection: local
  gather_facts: false
  tags: [rancher]
  vars:
    cp_ip: "{{ hostvars[groups['k3s_server']][0]].ansible_default_ipv4.address }}"
    # Rancher hostname: use the override if set, otherwise auto-pick nip.io of CP IP.
    rancher_host: "{{ rancher_hostname if rancher_hostname else 'rancher.' ~ cp_ip ~ '.nip.io' }}"
  tasks:
    - name: Check helm is on PATH
      ansible.builtin.command: helm version --short
      changed_when: false
      register: helm_check
      failed_when: helm_check.rc != 0

    - name: Add Helm repos
      kubernetes.core.helm_repository:
        name: "{{ item.name }}"
        repo_url: "{{ item.url }}"
      loop:
        - { name: jetstack, url: "https://charts.jetstack.io" }
        - { name: ingress-nginx, url: "https://kubernetes.github.io/ingress-nginx" }
        - { name: rancher-stable, url: "https://releases.rancher.com/server-charts/stable" }

    - name: Install ingress-nginx
      kubernetes.core.helm:
        name: ingress-nginx
        chart_ref: ingress-nginx/ingress-nginx
        release_namespace: ingress-nginx
        create_namespace: true
        kubeconfig: "{{ kubeconfig_local_path }}"
        wait: true
        timeout: 5m
        values:
          controller:
            service:
              type: LoadBalancer
          publishService:
            enabled: true

    - name: Install cert-manager (with CRDs)
      kubernetes.core.helm:
        name: cert-manager
        chart_ref: jetstack/cert-manager
        release_namespace: cert-manager
        create_namespace: true
        kubeconfig: "{{ kubeconfig_local_path }}"
        wait: true
        timeout: 5m
        values:

```

```

    installCRDs: true

- name: Install Rancher
  kubernetes.core.helm:
    name: rancher
    chart_ref: rancher-stable/rancher
    chart_version: "{{ rancher_chart_version | default(omit, true) }}"
    release_namespace: "{{ rancher_namespace }}"
    create_namespace: true
    kubeconfig: "{{ kubeconfig_local_path }}"
    wait: true
    timeout: 15m
    values:
      hostname: "{{ rancher_host }}"
      bootstrapPassword: "{{ rancher_bootstrap_password }}"
      replicas: 1
      ingress:
        ingressClassName: nginx

- name: Wait for Rancher rollout
  kubernetes.core.k8s_info:
    kubeconfig: "{{ kubeconfig_local_path }}"
    api_version: apps/v1
    kind: Deployment
    name: rancher
    namespace: "{{ rancher_namespace }}"
  register: rancher_deploy
  until:
    - rancher_deploy.resources | length > 0
    - rancher_deploy.resources[0].status.readyReplicas | default(0) >= 1
  retries: 90
  delay: 10

- name: Print access info
  ansible.builtin.debug:
    msg:
      - "Rancher ready at: https://{{ rancher_host }}"
      - "Bootstrap password: {{ rancher_bootstrap_password }}"
      - "Accept the self-signed cert, then set a real admin password on the welcome screen."

```